

TALLER ANALÍTICO: CALCULANDO EL DISPARO (25 MIN)

Tenemos un Perceptrón diseñado para decidir si debemos aprobar un crédito (Salida 1) o rechazarlo (Salida 0). Recibe dos entradas (Ingresos= X_1 , Deudas= X_2). Los pesos internos de la neurona son $W_1 = 0.8$ y $W_2 = -0.5$. El sesgo de la neurona es $b = -10$.

Evaluemos a un cliente: Sus ingresos (X_1) son 50, y sus deudas (X_2) son 20.

1. Calcule matemáticamente el valor Z usando la ecuación de combinación lineal.
2. Pase el valor Z calculado por la Función Escalón. ¿La neurona "disparará" aprobando el crédito (1) o no (0)?
3. Analice los pesos: ¿Por qué tiene sentido que W_2 sea negativo en este escenario empresarial?

1.
$$Z = (X_1 \times W_1) + (X_2 \times W_2) + b$$
$$Z = (50 \times 0.8) + (20 \times -0.5) + (-10)$$
$$Z = 40 + (-10) + (-10)$$
$$Z = 40 - 10 - 10$$
$$Z = 20$$

2. La regla dice: si Z es mayor o igual a 0, la salida es 1. Si Z es menor a 0, la salida es 0
Como $Z = 20$, y 20 es mayor que 0:
La salida es 1 - la neurona SÍ dispara, el crédito es APROBADO

3. W_2 es el peso que acompaña a las deudas. Su valor es negativo (-0.5) porque las deudas afectan de forma negativa la posibilidad de aprobar un crédito. Es decir, mientras más deudas tenga una persona, menos puntos obtiene en la decisión final.
Por otro lado, los ingresos tienen un peso positivo (0.8), porque mayores ingresos aumentan la posibilidad de aprobación.
El sesgo ($b = -10$) establece un nivel mínimo que se debe superar para que el crédito sea aprobado

TALLER DE LABORATORIO: HACKEANDO LOS PESOS (35 MIN)**Misión Práctica: La Compuerta OR**

1. Transcriban el código del perceptrón a su IDE.
2. El código actual resuelve la compuerta AND (solo da 1 si le pasan [1, 1]).
Verifiquen pasando las entradas [1, 0], [0, 1] y [0, 0]; todas deberían arrojar 0.
3. **El Reto:** Modifiquen MANUALMENTE el valor del vector pesos (W) y/o el sesgo (b). Su objetivo es encontrar la combinación matemática exacta para que la neurona resuelva la **Compuerta OR**.
4. Reglas del OR: Debe arrojar 1 si las entradas son [1, 1], [1, 0] o [0, 1]. Solo debe arrojar 0 si la entrada es [0, 0].
5. Anoten los pesos y el sesgo que lograron resolver el problema. Acaban de hacer lo que en la próxima clase la red neuronal hará sola: ¡Ajustar sus pesos (entrenar)!

```
Problems Output Debug Console Terminal Ports 1
(.venv) @gomezzuley19-debug →/workspaces/IA-2-Talleres (main) $ /workspaces/IA-2-Talleres/.venv/bin/python "/workspaces/IA-2-Talleres/main.py"
[[3. 3.]
 [4. 2.]
 [6. 6.]]
El punto [5,4] pertenece a la clase: 0

===== PARTE 2: AGREGANDO EL PUNTO 'PROBLEMÁTICO' [5,5] =====

--- Modelo LINEAL con el nuevo punto ---
Vectores de Soporte:
[[5. 5.]
 [6. 6.]]
Precisión del modelo lineal: 1.0
Predicción para [5,5]: 0 (debería ser 0)

===== PARTE 3: CAMBIANDO A KERNEL RBF =====

--- Modelo RBF con el nuevo punto ---
Vectores de Soporte:
[[2. 2.]
 [4. 2.]
 [5. 5.]
 [6. 6.]
 [7. 8.]
 [8. 7.]]
Precisión del modelo RBF: 1.0
Predicción para [5,5]: 0 (debería ser 0)
(.venv) @gomezzuley19-debug →/workspaces/IA-2-Talleres (main) $ /workspaces/IA-2-Talleres/.venv/bin/python /workspaces/IA-2-Talleres/main.py
===== COMPUERTA AND =====
Entrada [1, 1] -> Salida: 1
Entrada [1, 0] -> Salida: 0
Entrada [0, 1] -> Salida: 0
Entrada [0, 0] -> Salida: 0
(.venv) @gomezzuley19-debug →/workspaces/IA-2-Talleres (main) $
```



```

@gozzzuley19-debug → /workspaces/IA-2-Talleres (main) $ source /workspaces/IA-2-Talleres/.venv/bin/activate
(.venv) @gozzzuley19-debug → /workspaces/IA-2-Talleres (main) $ /workspaces/IA-2-Talleres/.venv/bin/python /workspaces,

===== COMPUERTA OR =====
Entrada [1, 1] -> Salida: 1
Entrada [1, 0] -> Salida: 1
Entrada [0, 1] -> Salida: 1
Entrada [0, 0] -> Salida: 0
(.venv) @gozzzuley19-debug → /workspaces/IA-2-Talleres (main) $

```

El sesgo (b) funciona como el "nivel de exigencia" de la neurona: es lo que hay que superar para que dispare un 1.

Con el AND ($b = -0.8$), la neurona era muy exigente. Si solo llegaba una entrada encendida, se sumaba 0.5, pero al restarle 0.8 el resultado quedaba negativo (-0.3), así que no disparaba. Solo cuando llegaban las dos entradas juntas ($0.5 + 0.5 = 1.0$) se superaba el 0.8 y la neurona daba 1.

Con el OR ($b = -0.3$), la neurona se volvió menos exigente. Ahora, con una sola entrada encendida basta: $0.5 - 0.3 = 0.2$, que es positivo, así que ya dispara. Y cuando llegan las dos, con más razón ($1.0 - 0.3 = 0.7$). Lo único que sigue dando 0 es cuando no llega ninguna entrada, porque $0 - 0.3 = -0.3$, que es negativo.

En resumen: bajar el sesgo es como bajarle el nivel de exigencia a la neurona. El AND pide "las dos cosas", el OR pide "al menos una". Y esto es exactamente lo que en la próxima clase la red neuronal hará automáticamente al entrenarse: ir ajustando estos números por sí sola hasta encontrar la combinación que resuelve el problema.